

06b__inverse__design__robust__refinement

April 27, 2026

1 06b - Inverse Design Robust Refinement

This notebook refines the first inverse-design solution.

The first inverse-design notebook found many feasible candidates, but the best 20 designs were concentrated near upper input bounds, especially **gravity** and **porosity**.

This notebook adds two improvements:

1. **Boundary-aware scoring:** candidates near input bounds receive a penalty.
2. **Local grid refinement:** after discovering promising regions, we create a structured local grid around those regions instead of relying only on random sampling.

The goal is not to replace the first inverse-design notebook, but to compare:

- **v1:** best score candidates from sampling;
- **v2:** robust candidates with boundary penalty;
- **v3:** local grid refined candidates.

Final output:

outputs/submissions/design_submission_robust_refined.csv

1.1 1. Imports and Project Paths

```
[1]: from pathlib import Path
import json
import itertools
import warnings

import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import ExtraTreesRegressor
from sklearn.preprocessing import MinMaxScaler

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)
```

```

PROJECT_ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.
↳ cwd()
FORWARD_DIR = PROJECT_ROOT / "data" / "raw" / "forward_prediction"
INVERSE_DIR = PROJECT_ROOT / "data" / "raw" / "inverse_design"
OUTPUTS_DIR = PROJECT_ROOT / "outputs"
SUBMISSIONS_DIR = OUTPUTS_DIR / "submissions"
MODELS_DIR = OUTPUTS_DIR / "models"
REPORTS_DIR = PROJECT_ROOT / "reports"

for path in [SUBMISSIONS_DIR, MODELS_DIR, REPORTS_DIR]:
    path.mkdir(parents=True, exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Submissions directory:", SUBMISSIONS_DIR)
print("Reports directory:", REPORTS_DIR)

```

Project root: /home/alouiyaz/projects/boom-challenge-ejecta-prediction
Submissions directory: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/submissions
Reports directory: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/reports

1.2 2. Load Data and Constraints

We use the same training data and `constraints.json` file as before.

```

[2]: input_cols = [
    "energy", "angle_rad", "coupling", "strength",
    "porosity", "gravity", "atmosphere", "shape_factor"
]

target_cols = [
    "P80", "fines_frac", "oversize_frac",
    "R95", "R50_fines", "R50_oversize"
]

fragmentation_targets = ["P80", "fines_frac", "oversize_frac"]
distance_targets = ["R95", "R50_fines", "R50_oversize"]

raw_train = pd.read_csv(FORWARD_DIR / "train.csv")[input_cols]
raw_test = pd.read_csv(FORWARD_DIR / "test.csv")[input_cols]
y = pd.read_csv(FORWARD_DIR / "train_labels.csv")[target_cols]

constraints_path = INVERSE_DIR / "constraints.json"
if not constraints_path.exists():
    constraints_path = PROJECT_ROOT / "constraints.json"

```

```

with open(constraints_path, "r") as f:
    constraints_data = json.load(f)

output_constraints = constraints_data["constraints"]
input_bounds = constraints_data["input_bounds"]

p80_min = output_constraints["p80_min"]
p80_max = output_constraints["p80_max"]
r95_max = output_constraints["r95_max"]
p80_center = (p80_min + p80_max) / 2

print("Train:", raw_train.shape)
print("Targets:", y.shape)
print("Constraints:")
display(pd.DataFrame([output_constraints]))
print("Input bounds:")
display(pd.DataFrame(input_bounds).T)

```

Train: (2930, 8)

Targets: (2930, 6)

Constraints:

	p80_min	p80_max	r95_max
0	96.0	101.0	175.0

Input bounds:

	min	max
energy	0.500000	5.000000
angle_rad	0.261799	1.570796
coupling	0.200000	1.700000
strength	0.400000	4.200000
porosity	0.000000	0.330000
gravity	1.020000	10.470000
atmosphere	0.000000	1.000000
shape_factor	0.700000	1.500000

1.3 3. Feature Engineering

This function must match the forward-model notebooks.

```

[3]: def add_physics_features(X: pd.DataFrame) -> pd.DataFrame:
    X = X.copy()
    eps = 1e-9

    X["effective_energy"] = X["energy"] * X["coupling"]
    X["log_energy"] = np.log1p(X["energy"])
    X["log_effective_energy"] = np.log1p(X["effective_energy"])

```

```

X["sin_angle"] = np.sin(X["angle_rad"])
X["cos_angle"] = np.cos(X["angle_rad"])
X["tan_angle"] = np.tan(X["angle_rad"])
X["horizontal_energy"] = X["effective_energy"] * X["cos_angle"]
X["vertical_energy"] = X["effective_energy"] * X["sin_angle"]
X["vertical_horizontal_ratio"] = X["vertical_energy"] /
↪(X["horizontal_energy"] + eps)

X["energy_per_strength"] = X["energy"] / (X["strength"] + eps)
X["effective_energy_per_strength"] = X["effective_energy"] / (X["strength"]
↪+ eps)
X["material_resistance_index"] = X["strength"] * (1 - X["porosity"])
X["fragmentation_index"] = X["effective_energy"] * X["porosity"] /
↪(X["strength"] + eps)
X["coupling_porosity"] = X["coupling"] * X["porosity"]
X["coupling_atmosphere"] = X["coupling"] * X["atmosphere"]
X["porosity_strength"] = X["porosity"] * X["strength"]

X["energy_per_gravity"] = X["energy"] / (X["gravity"] + eps)
X["effective_energy_per_gravity"] = X["effective_energy"] / (X["gravity"] +
↪eps)
X["horizontal_energy_per_gravity"] = X["horizontal_energy"] / (X["gravity"]
↪+ eps)
X["vertical_energy_per_gravity"] = X["vertical_energy"] / (X["gravity"] +
↪eps)

X["drag_proxy"] = X["atmosphere"] * X["shape_factor"]
X["drag_per_gravity"] = X["drag_proxy"] / (X["gravity"] + eps)
X["atmosphere_shape_energy"] = X["atmosphere"] * X["shape_factor"] *
↪X["effective_energy"]
X["atmosphere_per_gravity"] = X["atmosphere"] / (X["gravity"] + eps)

X["pi_gravity_proxy"] = (X["gravity"] * X["coupling"]) / (X["energy"] + eps)
X["pi_strength_proxy"] = X["strength"] / (X["gravity"] * X["coupling"] +
↪eps)
X["pi_atmosphere_proxy"] = X["atmosphere"] / (X["gravity"] * X["coupling"]
↪+ eps)

X["porosity_regime"] = (X["porosity"] > 0.15).astype(int)
X["strength_regime"] = (X["strength"] > 2.6).astype(int)
X["angle_regime"] = (X["angle_rad"] > 0.95).astype(int)
X["atm_regime"] = (X["atmosphere"] > 0.30).astype(int)
X["regime_combo"] = (
    X["porosity_regime"] * 8
    + X["strength_regime"] * 4
    + X["angle_regime"] * 2

```

```

        + X["atm_regime"]
    )

    X["scaled_energy"] = X["effective_energy"] / (X["strength"] * np.
↪sqrt(X["gravity"]) + eps)
    X["fragility"] = X["porosity"] / (X["strength"] + eps)
    X["range_proxy"] = (X["effective_energy"] * (X["cos_angle"] ** 2)) / ↪
↪(X["gravity"] * X["strength"] + eps)
    X["energy_sin_angle"] = X["energy"] * X["sin_angle"]
    X["momentum_proxy"] = X["effective_energy"] * X["sin_angle"]

    X["coupling_per_atm_clipped"] = X["coupling"] / (X["atmosphere"] + 1e-3)
    X["log_coupling_per_atm"] = np.log1p(X["coupling_per_atm_clipped"])
    X["retention_factor"] = X["atmosphere"] * X["drag_proxy"] / (X["energy"] + ↪
↪eps)

    return X

X_fe_train = add_physics_features(raw_train)
print("X_fe_train:", X_fe_train.shape)

```

X_fe_train: (2930, 48)

1.4 4. Feature Sets and Model Builders

We need these definitions to reload the saved forward model and to train the ensemble models for P80 and R95.

```

[4]: raw_features = input_cols.copy()

core_physics_features = raw_features + [
    "effective_energy", "log_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "tan_angle",
    "horizontal_energy", "vertical_energy", "vertical_horizontal_ratio",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy", ↪
↪"atmosphere_per_gravity",
]

extended_physics_features = core_physics_features + [
    "pi_gravity_proxy", "pi_strength_proxy", "pi_atmosphere_proxy",
    "scaled_energy", "fragility", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "coupling_per_atm_clipped", "log_coupling_per_atm", "retention_factor",

```

```

]

extended_plus_regimes_features = extended_physics_features + [
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo"
]

fragmentation_features = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_strength", "effective_energy_per_strength",
    "material_resistance_index", "fragmentation_index",
    "coupling_porosity", "coupling_atmosphere", "porosity_strength",
    "drag_proxy", "atmosphere_shape_energy",
    "pi_strength_proxy", "pi_atmosphere_proxy", "scaled_energy", "fragility",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo",
]

distance_features = raw_features + [
    "effective_energy", "log_effective_energy",
    "sin_angle", "cos_angle", "horizontal_energy", "vertical_energy",
    "energy_per_gravity", "effective_energy_per_gravity",
    "horizontal_energy_per_gravity", "vertical_energy_per_gravity",
    "drag_proxy", "drag_per_gravity", "atmosphere_shape_energy",
    ↪ "atmosphere_per_gravity",
    "pi_gravity_proxy", "pi_atmosphere_proxy", "scaled_energy", "range_proxy",
    "energy_sin_angle", "momentum_proxy",
    "porosity_regime", "strength_regime", "angle_regime", "atm_regime",
    ↪ "regime_combo",
]

def clean_feature_list(features, X):
    seen = set()
    out = []
    for f in features:
        if f in X.columns and f not in seen:
            out.append(f)
            seen.add(f)
    return out

raw_features = clean_feature_list(raw_features, X_fe_train)
core_physics_features = clean_feature_list(core_physics_features, X_fe_train)
extended_physics_features = clean_feature_list(extended_physics_features,
    ↪ X_fe_train)

```

```

extended_plus_regimes_features =
    ↪ clean_feature_list(extended_plus_regimes_features, X_fe_train)
fragmentation_features = clean_feature_list(fragmentation_features, X_fe_train)
distance_features = clean_feature_list(distance_features, X_fe_train)
full_features = X_fe_train.columns.tolist()

feature_sets = {
    "raw": raw_features,
    "core_physics": core_physics_features,
    "extended_physics": extended_physics_features,
    "extended_plus_regimes": extended_plus_regimes_features,
    "fragmentation": fragmentation_features,
    "distance": distance_features,
    "full_features": full_features,
}

def get_features_for_feature_set(feature_set_name: str, target: str):
    if feature_set_name == "target_family_specific":
        return fragmentation_features if target in fragmentation_targets else
        ↪ distance_features
    return feature_sets[feature_set_name]

def build_extratrees(random_state=42):
    return ExtraTreesRegressor(
        n_estimators=800,
        max_features="sqrt",
        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )

try:
    from catboost import CatBoostRegressor
    def build_catboost(random_state=42):
        return CatBoostRegressor(
            iterations=1200,
            learning_rate=0.03,
            depth=6,
            l2_leaf_reg=5.0,
            loss_function="RMSE",
            random_seed=random_state,
            verbose=False,
        )
    CATBOOST_AVAILABLE = True
except Exception as e:
    print("CatBoost unavailable:", e)
    CATBOOST_AVAILABLE = False

```

```

try:
    from xgboost import XGBRegressor
    def build_xgb(random_state=42):
        return XGBRegressor(
            n_estimators=900,
            learning_rate=0.03,
            max_depth=4,
            min_child_weight=2,
            subsample=0.85,
            colsample_bytree=0.85,
            reg_alpha=0.05,
            reg_lambda=2.0,
            objective="reg:squarederror",
            random_state=random_state,
            n_jobs=-1,
        )
    XGB_AVAILABLE = True
except Exception as e:
    print("XGBoost unavailable:", e)
    XGB_AVAILABLE = False

try:
    from lightgbm import LGBMRegressor
    def build_lgbm(random_state=42):
        return LGBMRegressor(
            n_estimators=1200,
            learning_rate=0.025,
            num_leaves=31,
            min_child_samples=20,
            subsample=0.85,
            colsample_bytree=0.85,
            reg_alpha=0.05,
            reg_lambda=2.0,
            random_state=random_state,
            n_jobs=-1,
            verbosity=-1,
        )
    LGBM_AVAILABLE = True
except Exception as e:
    print("LightGBM unavailable:", e)
    LGBM_AVAILABLE = False

print("CatBoost:", CATBOOST_AVAILABLE, "XGBoost:", XGB_AVAILABLE, "LightGBM:",
      LGBM_AVAILABLE)

```

CatBoost: True XGBoost: True LightGBM: True

1.5 5. Load the Final Forward Model

The saved model from notebook 05 is used as the main surrogate simulator.

```
[5]: def clip_predictions(preds, target):
    preds = np.asarray(preds).copy()
    if target in ["fines_frac", "oversize_frac"]:
        return np.clip(preds, 0, 1)
    return np.clip(preds, 0, None)

model_builders = {"ExtraTrees": build_extratrees}
if CATBOOST_AVAILABLE:
    model_builders["CatBoost"] = build_catboost

class TargetSpecificForwardModel:
    def __init__(self, target_configs, feature_engineering_func,
        feature_selector, random_state=42):
        self.target_configs = target_configs
        self.feature_engineering_func = feature_engineering_func
        self.feature_selector = feature_selector
        self.random_state = random_state
        self.models_ = {}
        self.feature_columns_ = None

    def fit(self, X_raw: pd.DataFrame, y_df: pd.DataFrame):
        X_fe = self.feature_engineering_func(X_raw)
        self.feature_columns_ = X_fe.columns.tolist()

        for i, target in enumerate(y_df.columns):
            config = self.target_configs[target]
            model_name = config["model"]
            feature_set_name = config["feature_set"]
            model_builder = model_builders[model_name]
            features = self.feature_selector(feature_set_name, target)
            model = model_builder(random_state=self.random_state + i)
            model.fit(X_fe[features], y_df[target])
            self.models_[target] = {
                "model": model,
                "model_name": model_name,
                "feature_set": feature_set_name,
                "features": features,
            }
        return self

    def predict(self, X_raw: pd.DataFrame) -> pd.DataFrame:
        X_fe = self.feature_engineering_func(X_raw)
        preds = pd.DataFrame(index=X_raw.index)
        for target, info in self.models_.items():
```

```

        model = info["model"]
        features = info["features"]
        pred = model.predict(X_fe[features])
        preds[target] = clip_predictions(pred, target)
    return preds[target_cols]

def describe(self):
    rows = []
    for target, info in self.models_.items():
        rows.append({
            "target": target,
            "model": info["model_name"],
            "feature_set": info["feature_set"],
            "n_features": len(info["features"]),
        })
    return pd.DataFrame(rows)

target_configs = {
    "P80": {"model": "ExtraTrees", "feature_set": "target_family_specific"},
    "fines_frac": {"model": "ExtraTrees", "feature_set": "
    ↪target_family_specific"},
    "oversize_frac": {
        "model": "CatBoost" if CATBOOST_AVAILABLE else "ExtraTrees",
        "feature_set": "extended_plus_regimes" if CATBOOST_AVAILABLE else "
    ↪target_family_specific",
    },
    "R95": {"model": "ExtraTrees", "feature_set": "extended_plus_regimes"},
    "R50_fines": {"model": "ExtraTrees", "feature_set": "
    ↪extended_plus_regimes"},
    "R50_oversize": {"model": "ExtraTrees", "feature_set": "
    ↪extended_plus_regimes"},
}

model_path = MODELS_DIR / "final_target_specific_forward_model.joblib"
try:
    final_forward_model = joblib.load(model_path)
    print("Loaded final forward model from:", model_path)
except Exception as e:
    print("Could not load saved final model. Rebuilding it from training data.")
    print("Reason:", e)
    final_forward_model = TargetSpecificForwardModel(
        target_configs=target_configs,
        feature_engineering_func=add_physics_features,
        feature_selector=get_features_for_feature_set,
        random_state=42,
    )
    final_forward_model.fit(raw_train, y)

```

```
display(final_forward_model.describe())
```

Loaded final forward model from: /home/alouiyaz/projects/boom-challenge-ejecta-prediction/outputs/models/final_target_specific_forward_model.joblib

	target	model	feature_set	n_features
0	P80	ExtraTrees	target_family_specific	34
1	finest_frac	ExtraTrees	target_family_specific	34
2	oversize_frac	CatBoost	extended_plus_regimes	48
3	R95	ExtraTrees	extended_plus_regimes	48
4	R50_fines	ExtraTrees	extended_plus_regimes	48
5	R50_oversize	ExtraTrees	extended_plus_regimes	48

1.6 6. Build Constraint Ensemble for P80 and R95

The final forward model is the main predictor, but the ensemble helps estimate uncertainty and avoid fragile candidates.

```
[6]: ensemble_specs = []
ensemble_specs.append(("ExtraTrees_family", build_extratrees,
    ↪ "target_family_specific"))
ensemble_specs.append(("ExtraTrees_regimes", build_extratrees,
    ↪ "extended_plus_regimes"))

if CATBOOST_AVAILABLE:
    ensemble_specs.append(("CatBoost_family", build_catboost,
    ↪ "target_family_specific"))
    ensemble_specs.append(("CatBoost_regimes", build_catboost,
    ↪ "extended_plus_regimes"))

if XGB_AVAILABLE:
    ensemble_specs.append(("XGBoost_family", build_xgb,
    ↪ "target_family_specific"))
    ensemble_specs.append(("XGBoost_regimes", build_xgb,
    ↪ "extended_plus_regimes"))

if LGBM_AVAILABLE:
    ensemble_specs.append(("LightGBM_family", build_lgbm,
    ↪ "target_family_specific"))
    ensemble_specs.append(("LightGBM_regimes", build_lgbm,
    ↪ "extended_plus_regimes"))

print("Ensemble specs:")
for name, _, fset in ensemble_specs:
    print("-", name, "|", fset)

constraint_targets = ["P80", "R95"]
```

```

ensemble_models = {target: [] for target in constraint_targets}

for target in constraint_targets:
    print(f"Training ensemble models for {target}...")
    for i, (name, builder, feature_set_name) in enumerate(ensemble_specs):
        features = get_features_for_feature_set(feature_set_name, target)
        model = builder(random_state=1000 + i)
        model.fit(X_fe_train[features], y[target])
        ensemble_models[target].append({
            "name": name,
            "model": model,
            "features": features,
            "feature_set": feature_set_name,
        })
    print(" trained", name)

```

Ensemble specs:

- ExtraTrees_family | target_family_specific
- ExtraTrees_regimes | extended_plus_regimes
- CatBoost_family | target_family_specific
- CatBoost_regimes | extended_plus_regimes
- XGBoost_family | target_family_specific
- XGBoost_regimes | extended_plus_regimes
- LightGBM_family | target_family_specific
- LightGBM_regimes | extended_plus_regimes

Training ensemble models for P80...

```

trained ExtraTrees_family
trained ExtraTrees_regimes
trained CatBoost_family
trained CatBoost_regimes
trained XGBoost_family
trained XGBoost_regimes
trained LightGBM_family
trained LightGBM_regimes

```

Training ensemble models for R95...

```

trained ExtraTrees_family
trained ExtraTrees_regimes
trained CatBoost_family
trained CatBoost_regimes
trained XGBoost_family
trained XGBoost_regimes
trained LightGBM_family
trained LightGBM_regimes

```

1.7 7. Load Candidate Pool from Notebook 06 or Recreate It

If the first inverse-design notebook was already executed, we reuse the saved candidate report.

If not available, we regenerate a smaller candidate pool here.

```

[7]: RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)

def bounds_arrays(input_bounds, input_cols):
    lower = np.array([input_bounds[c]["min"] for c in input_cols], dtype=float)
    upper = np.array([input_bounds[c]["max"] for c in input_cols], dtype=float)
    return lower, upper

lower_bounds, upper_bounds = bounds_arrays(input_bounds, input_cols)

def sample_latin_hypercube(n_samples: int, random_state: int = 42) -> pd.
    DataFrame:
    try:
        from scipy.stats import qmc
        sampler = qmc.LatinHypercube(d=len(input_cols), seed=random_state)
        sample = sampler.random(n_samples)
        values = qmc.scale(sample, lower_bounds, upper_bounds)
    except Exception as e:
        print("Latin Hypercube unavailable. Falling back to uniform sampling.")
        values = rng.uniform(lower_bounds, upper_bounds, size=(n_samples,
    len(input_cols)))
    return pd.DataFrame(values, columns=input_cols)

def feasibility_mask(df_targets):
    return (df_targets["P80"].between(p80_min, p80_max)) & (df_targets["R95"]
    <= r95_max)

def near_feasible_mask(df_targets):
    return (df_targets["P80"].between(80, 120)) & (df_targets["R95"] <= 250)

true_feasible_train = feasibility_mask(y)
near_feasible_train = near_feasible_mask(y)

print("True feasible train rows:", int(true_feasible_train.sum()))
print("Near feasible train rows:", int(near_feasible_train.sum()))

candidate_report_path = REPORTS_DIR / "inverse_design_candidate_pool.csv"

if candidate_report_path.exists():
    print("Loading existing candidate report:", candidate_report_path)
    previous_candidates = pd.read_csv(candidate_report_path)
    # The previous report already contains predictions and score for top 20k
    candidates.
    # We reuse its input columns as a high-quality candidate seed pool.
    seed_candidates = previous_candidates[input_cols].drop_duplicates().
    reset_index(drop=True)
    print("Seed candidates:", seed_candidates.shape)

```

```

else:
    print("Existing candidate report not found. Regenerating a candidate seed_
    ↪pool.")
    seed_candidates = sample_latin_hypercube(150_000, random_state=RANDOM_STATE)
    print("Seed candidates:", seed_candidates.shape)

```

```

True feasible train rows: 35
Near feasible train rows: 372
Loading existing candidate report: /home/alouiyaz/projects/boom-challenge-
ejecta-prediction/reports/inverse_design_candidate_pool.csv
Seed candidates: (20000, 8)

```

1.8 8. Local Grid Refinement

Instead of searching the full 8-dimensional space with a coarse grid, we create a **local grid** around promising candidate ranges.

This answers the idea: *why not use a list of choices over intervals instead of random values?*

We use the previous candidate pool to identify a promising region, then build a structured grid inside that local region.

```

[8]: def create_local_intervals_from_seed(seed_df, input_cols, input_bounds, q_low=0.
    ↪05, q_high=0.95, padding_ratio=0.03):
    intervals = {}
    for col in input_cols:
        low_allowed = input_bounds[col]["min"]
        high_allowed = input_bounds[col]["max"]
        width_allowed = high_allowed - low_allowed

        local_low = seed_df[col].quantile(q_low) - padding_ratio * width_allowed
        local_high = seed_df[col].quantile(q_high) + padding_ratio *
    ↪width_allowed

        local_low = max(low_allowed, local_low)
        local_high = min(high_allowed, local_high)

        intervals[col] = (float(local_low), float(local_high))
    return intervals

local_intervals = create_local_intervals_from_seed(
    seed_candidates,
    input_cols=input_cols,
    input_bounds=input_bounds,
    q_low=0.05,
    q_high=0.95,
    padding_ratio=0.02,
)

```

```
pd.DataFrame(local_intervals, index=["local_min", "local_max"]).T
```

```
[8]:
```

	local_min	local_max
energy	2.693512	4.585454
angle_rad	0.501950	0.885743
coupling	0.536660	1.573947
strength	0.955697	2.506637
porosity	0.251777	0.330000
gravity	8.922212	10.470000
atmosphere	0.394378	0.864865
shape_factor	0.759789	1.333853

```
[9]: def make_local_grid(local_intervals, points_per_feature=5):
    grid_values = {
        col: np.linspace(low, high, points_per_feature)
        for col, (low, high) in local_intervals.items()
    }
    grid = pd.DataFrame(
        itertools.product(*grid_values.values()),
        columns=grid_values.keys(),
    )
    return grid

# 5^8 = 390,625 candidates. This is usually manageable.
# If execution is too long, reduce to 4. If your machine is fast, test 6.
POINTS_PER_FEATURE = 5
local_grid_candidates = make_local_grid(local_intervals,
    points_per_feature=POINTS_PER_FEATURE)

print("Local grid candidates:", local_grid_candidates.shape)
display(local_grid_candidates.head())
```

Local grid candidates: (390625, 8)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	shape_factor
0	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	0.759789
1	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	0.903305
2	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	1.046821
3	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	1.190337
4	2.693512	0.50195	0.53666	0.955697	0.251777	8.922212	0.394378	1.333853

1.9 9. Combine Candidate Sources

We combine:

- high-quality candidates from previous sampling;
- local grid candidates;
- near-feasible training anchors.

```
[10]: anchor_candidates = raw_train.loc[near_feasible_train, input_cols].copy()

combined_candidates = pd.concat(
    [seed_candidates, local_grid_candidates, anchor_candidates],
    ignore_index=True,
)
combined_candidates = combined_candidates.round(10).drop_duplicates().
    ↪reset_index(drop=True)

print("Combined candidates:", combined_candidates.shape)
display(combined_candidates.head())
```

Combined candidates: (410949, 8)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	↪
	↪shape_factor							
0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	0.486195	↪
	↪1.321979							
1	3.801488	0.567512	0.757391	1.385881	0.294003	10.179310	0.839499	↪
	↪0.832437							
2	3.217315	0.742617	1.333629	1.811721	0.306086	10.455045	0.664532	↪
	↪0.916205							
3	3.223864	0.625563	0.864345	1.346453	0.330000	9.989450	0.616066	↪
	↪1.024933							
4	3.857398	0.636890	0.847146	1.632682	0.330000	10.030104	0.822272	↪
	↪1.166845							

1.10 10. Predict Candidate Outcomes

```
[11]: def predict_constraint_ensemble(candidates_raw: pd.DataFrame) -> pd.DataFrame:
    X_fe = add_physics_features(candidates_raw)
    out = pd.DataFrame(index=candidates_raw.index)

    for target in constraint_targets:
        model_preds = []
        for info in ensemble_models[target]:
            pred = info["model"].predict(X_fe[info["features"]])
            pred = clip_predictions(pred, target)
            model_preds.append(pred)
            out[f"{target}_{info['name']}"] = pred
```



```

    pred_matrix = np.vstack(model_preds).T
    out[f"{target}_ens_mean"] = pred_matrix.mean(axis=1)
    out[f"{target}_ens_std"] = pred_matrix.std(axis=1)
    out[f"{target}_ens_min"] = pred_matrix.min(axis=1)
    out[f"{target}_ens_max"] = pred_matrix.max(axis=1)

    return out

print("Predicting with final forward model...")
final_preds = final_forward_model.predict(combined_candidates)

print("Predicting with constraint ensemble...")
ensemble_preds = predict_constraint_ensemble(combined_candidates)

candidate_results = pd.concat(
    [combined_candidates, final_preds.add_prefix("final_"), ensemble_preds],
    axis=1,
)

print("Candidate results:", candidate_results.shape)
display(candidate_results.head())

```

Predicting with final forward model...

Predicting with constraint ensemble...

Candidate results: (410949, 38)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	
	shape_factor	final_P80	final_fines_frac	final_oversize_frac	final_R95			
	final_R50_fines							
0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	0.486195	
	1.321979	98.139903		0.095237		0.070884	86.592805	
	81.897310							
1	3.801488	0.567512	0.757391	1.385881	0.294003	10.179310	0.839499	
	0.832437	99.075405		0.079425		0.064659	74.806389	
	72.547138							
2	3.217315	0.742617	1.333629	1.811721	0.306086	10.455045	0.664532	
	0.916205	98.038523		0.095039		0.066172	89.890288	
	86.256122							
3	3.223864	0.625563	0.864345	1.346453	0.330000	9.989450	0.616066	
	1.024933	99.701490		0.081914		0.076078	77.821359	
	76.548422							
4	3.857398	0.636890	0.847146	1.632682	0.330000	10.030104	0.822272	
	1.166845	99.145625		0.080559		0.083974	76.643641	
	72.577342							
	final_R50_oversize	P80_ExtraTrees_family	P80_ExtraTrees_regimes					
	P80_CatBoost_family	P80_CatBoost_regimes	P80_XGBoost_family					
	P80_XGBoost_regimes	P80_LightGBM_family						

0	37.264893	97.955118	98.136411	□
↪	98.511066	98.026610	98.959793	98.932312 □
↪	98.516665			
1	31.481105	98.514339	98.560923	□
↪	97.978788	98.407702	97.882523	97.932083 □
↪	99.319454			
2	39.213811	97.997170	98.583399	□
↪	98.041263	98.808573	98.468315	98.692719 □
↪	99.244739			
3	34.694682	99.833900	99.130907	□
↪	97.853357	98.208421	98.492271	98.088333 □
↪	98.128746			
4	34.484685	98.962234	99.113658	□
↪	98.615458	99.036964	98.563988	98.448174 □
↪	97.325681			

	P80_LightGBM_regimes	P80_ens_mean	P80_ens_std	P80_ens_min	P80_ens_max	□
↪	R95_ExtraTrees_family	R95_ExtraTrees_regimes	R95_CatBoost_family			□
↪	R95_CatBoost_regimes	\				
0	98.899820	98.492224	0.389112	97.955118	98.959793	□
↪	85.534724	87.082336		81.315724		85.
↪	577557					
1	99.167034	98.470356	0.512020	97.882523	99.319454	□
↪	75.005754	74.203698		80.464459		79.
↪	141923					
2	98.429498	98.533209	0.379865	97.997170	99.244739	□
↪	87.940196	91.585254		78.595402		87.
↪	395893					
3	98.117837	98.481722	0.625623	97.853357	99.833900	□
↪	79.502793	77.325436		84.589988		77.
↪	668675					
4	98.307637	98.546724	0.535868	97.325681	99.113658	□
↪	75.954197	75.324401		75.386544		75.
↪	557568					

	R95_XGBoost_family	R95_XGBoost_regimes	R95_LightGBM_family	□
↪	R95_LightGBM_regimes	R95_ens_mean	R95_ens_std	R95_ens_min R95_ens_max
0	93.574913	88.822227		88.515601 84.
↪	883030	86.913264	3.349563	81.315724 93.574913
1	77.671082	83.101456		70.914273 75.
↪	433745	76.992049	3.632380	70.914273 83.101456
2	86.542168	84.776237		83.924809 84.
↪	496936	85.657112	3.520522	78.595402 91.585254
3	82.663376	81.344727		82.414974 72.
↪	334758	79.730591	3.653985	72.334758 84.589988

4	75.962784	80.362534	77.377862	74.
679235	76.325641	1.690351	74.679235	80.362534

1.11 11. Boundary Penalty and Robust Score

This section implements the idea of penalizing candidates that are too close to the allowed input bounds.

The penalty is not a percentage. It is a coefficient added to the design objective.

```
[12]: def constraint_violation(p80, r95, p80_min=p80_min, p80_max=p80_max,
    ↪r95_max=r95_max):
    return (
        np.maximum(p80_min - p80, 0)
        + np.maximum(p80 - p80_max, 0)
        + np.maximum(r95 - r95_max, 0)
    )

    candidate_results["final_feasible"] = (
        candidate_results["final_P80"].between(p80_min, p80_max)
        & (candidate_results["final_R95"] <= r95_max)
    )

    candidate_results["ens_feasible"] = (
        candidate_results["P80_ens_mean"].between(p80_min, p80_max)
        & (candidate_results["R95_ens_mean"] <= r95_max)
    )

    candidate_results["safe_feasible"] = (
        candidate_results["P80_ens_mean"].between(97.0, 100.0)
        & (candidate_results["R95_ens_mean"] <= 160.0)
        & (candidate_results["final_P80"].between(96.0, 101.0))
        & (candidate_results["final_R95"] <= 175.0)
    )

    candidate_results["ens_constraint_violation"] = constraint_violation(
        candidate_results["P80_ens_mean"],
        candidate_results["R95_ens_mean"],
    )

    candidate_results["final_constraint_violation"] = constraint_violation(
        candidate_results["final_P80"],
        candidate_results["final_R95"],
    )

    candidate_results["design_score"] = (
        8.0 * np.abs(candidate_results["P80_ens_mean"] - p80_center)
        + 0.04 * candidate_results["R95_ens_mean"]
```

```

+ 1.5 * candidate_results["P80_ens_std"]
+ 0.04 * candidate_results["R95_ens_std"]
+ 0.20 * candidate_results["energy"]
+ 15.0 * candidate_results["ens_constraint_violation"]
+ 15.0 * candidate_results["final_constraint_violation"]
)

summary_df = pd.DataFrame([
    "total_candidates": len(candidate_results),
    "final_feasible": int(candidate_results["final_feasible"].sum()),
    "ensemble_feasible": int(candidate_results["ens_feasible"].sum()),
    "safe_feasible": int(candidate_results["safe_feasible"].sum()),
])

display(summary_df)

```

	total_candidates	final_feasible	ensemble_feasible	safe_feasible
0	410949	39139	35445	18095

```

[13]: def boundary_penalty(df, input_cols, input_bounds, margin_ratio=0.05):
    penalty = np.zeros(len(df))
    details = {}

    for col in input_cols:
        low = input_bounds[col]["min"]
        high = input_bounds[col]["max"]
        width = high - low
        margin = margin_ratio * width

        distance_to_low = df[col] - low
        distance_to_high = high - df[col]

        low_penalty = np.maximum(margin - distance_to_low, 0) / (margin + 1e-12)
        high_penalty = np.maximum(margin - distance_to_high, 0) / (margin +
↪1e-12)

        col_penalty = low_penalty + high_penalty
        penalty += col_penalty
        details[f"boundary_penalty_{col}"] = col_penalty

    details_df = pd.DataFrame(details, index=df.index)
    return penalty, details_df

boundary_weight = 3.0

penalty, penalty_details = boundary_penalty(
    candidate_results,

```

```

        input_cols=input_cols,
        input_bounds=input_bounds,
        margin_ratio=0.05,
    )

    candidate_results["boundary_penalty"] = penalty
    candidate_results = pd.concat([candidate_results, penalty_details], axis=1)

    candidate_results["robust_design_score"] = (
        candidate_results["design_score"]
        + boundary_weight * candidate_results["boundary_penalty"]
    )

    safe_pool = candidate_results[candidate_results["safe_feasible"]].copy()
    candidate_pool = candidate_results[
        candidate_results["final_feasible"] & candidate_results["ens_feasible"]
    ].copy()

    print("Candidate pool:", candidate_pool.shape)
    print("Safe pool:", safe_pool.shape)

    display(
        safe_pool.sort_values("robust_design_score").head(10)[
            input_cols + [
                "final_P80", "final_R95", "P80_ens_mean", "P80_ens_std",
                "R95_ens_mean", "R95_ens_std",
                "design_score", "boundary_penalty", "robust_design_score"
            ]
        ]
    )

```

Candidate pool: (27995, 54)

Safe pool: (18095, 54)

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	
	shape_factor	final_P80	final_R95	P80_ens_mean	P80_ens_std	R95_ens_mean	R95_ens_std	\
0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	0.486195	
	1.321979	98.139903	86.592805	98.492224	0.389112	86.913264	3.349563	
5	2.846707	0.590529	1.080326	1.378950	0.282806	9.901995	0.683393	
	1.159860	97.934175	86.231628	98.483203	0.572645	86.495709	1.398957	
9	3.857509	0.625831	0.795776	1.514135	0.309255	9.938662	0.841641	
	1.181244	98.410902	72.894158	98.472878	0.722666	76.807189	3.376023	

11	2.840497	0.686096	0.841353	1.134742	0.283432	9.943441	0.867710	4.
	0.953802	97.122506	84.213834	98.444504	0.682039	78.768877		
	163790							
13	3.547463	0.693527	1.513037	2.183828	0.291489	9.972195	0.663254	5.
	1.069653	97.696331	83.201911	98.555360	0.547707	80.294774		
	013393							
16	3.600797	0.511221	1.264328	1.922918	0.303837	10.008936	0.617995	4.
	1.245303	98.181368	85.413788	98.506245	0.719625	85.285994		
	275933							
37	3.287377	0.851607	0.624994	1.003748	0.269562	9.897003	0.624273	3.
	1.219199	99.514990	80.321074	98.474173	0.895484	83.011685		
	564213							
42	3.870405	0.591411	0.789338	1.501928	0.313308	9.784023	0.836029	16.
	0.805102	98.322733	78.754366	98.464169	0.363311	87.476221		
	184090							
48	2.947683	0.508049	1.448914	1.767410	0.290708	9.937267	0.652395	3.
	1.170228	97.537812	81.666734	98.491711	0.949540	88.635690		
	769461							
23	3.578185	0.509034	1.291639	1.985480	0.308735	10.036174	0.598355	4.
	1.238038	98.967367	85.162695	98.517315	0.801667	83.114735		
	129823							

	design_score	boundary_penalty	robust_design_score
0	4.939725	0.000000	4.939725
5	5.078469	0.000000	5.078469
9	5.279804	0.000000	5.279804
11	5.352436	0.000000	5.352436
13	5.386258	0.000000	5.386258
16	5.432034	0.024202	5.504640
37	5.670350	0.000000	5.670350
42	5.752111	0.000000	5.752111
48	5.776367	0.000000	5.776367
23	5.546438	0.081850	5.791989

1.12 12. Diversified Selection

We select 20 designs using greedy diversity in scaled input space.

We compare:

- selection by `design_score`;
- selection by `robust_design_score`.

```
[14]: def greedy_diverse_selection(pool: pd.DataFrame, score_col: str, n_select: int,
    => 20, min_distance: float = 0.12) -> pd.DataFrame:
    if len(pool) == 0:
        return pool.copy()
```

```

pool_sorted = pool.sort_values(score_col).reset_index(drop=True)
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(pool_sorted[input_cols])

selected_indices = []
for i in range(len(pool_sorted)):
    if len(selected_indices) == 0:
        selected_indices.append(i)
    else:
        distances = np.linalg.norm(X_scaled[i] -
X_scaled[selected_indices], axis=1)
        if distances.min() >= min_distance:
            selected_indices.append(i)

    if len(selected_indices) >= n_select:
        break

    return pool_sorted.iloc[selected_indices].copy()

def select_20(pool, score_col):
    selected = pd.DataFrame()
    for threshold in [0.22, 0.20, 0.18, 0.15, 0.12, 0.10, 0.08, 0.05, 0.0]:
        selected = greedy_diverse_selection(pool, score_col=score_col,
n_select=20, min_distance=threshold)
        print(score_col, "threshold", threshold, "selected", len(selected))
        if len(selected) >= 20:
            break
    return selected.head(20).reset_index(drop=True)

selection_source = safe_pool if len(safe_pool) >= 20 else candidate_pool
print("Selection source:", selection_source.shape)

selected_v1_like = select_20(selection_source, score_col="design_score")
selected_robust = select_20(selection_source, score_col="robust_design_score")

print("Selected v1-like:", selected_v1_like.shape)
print("Selected robust:", selected_robust.shape)

```

```

Selection source: (18095, 54)
design_score threshold 0.22 selected 20
robust_design_score threshold 0.22 selected 20
Selected v1-like: (20, 54)
Selected robust: (20, 54)

```

1.13 13. Compare v1-like and Robust Selection

```
[15]: compare_cols = input_cols + [
    "final_P80", "final_R95", "final_fines_frac", "final_oversize_frac",
    "final_R50_fines", "final_R50_oversize",
    "P80_ens_mean", "P80_ens_std", "R95_ens_mean", "R95_ens_std",
    "design_score", "boundary_penalty", "robust_design_score",
]

def summarize_selection(df, name):
    summary = df[compare_cols].describe().T
    summary.insert(0, "selection", name)
    return summary

selection_summary = pd.concat([
    summarize_selection(selected_v1_like, "v1_like"),
    summarize_selection(selected_robust, "robust"),
])

display(selection_summary)

print("Robust selected designs:")
display(selected_robust[compare_cols])
```

	50%	selection	count	mean	std	min	25%
		75%	max				
energy		v1_like	20.0	3.541637	0.383221	2.840497	3.283104
	↪ 3.574130	3.761277	4.373093				
angle_rad		v1_like	20.0	0.667411	0.098041	0.501950	0.609381
	↪ 0.652797	0.753057	0.827086				
coupling		v1_like	20.0	1.116101	0.305705	0.545276	0.845698
	↪ 1.169042	1.342959	1.612882				
strength		v1_like	20.0	1.730795	0.379712	1.134742	1.370826
	↪ 1.731167	1.993796	2.358284				
porosity		v1_like	20.0	0.302809	0.020461	0.271333	0.287954
	↪ 0.299614	0.325351	0.330000				
gravity		v1_like	20.0	10.158141	0.213961	9.861554	9.985136
	↪ 10.137317	10.333350	10.470000				
atmosphere		v1_like	20.0	0.702251	0.145598	0.445090	0.613727
	↪ 0.673962	0.847732	0.872075				
shape_factor		v1_like	20.0	1.067431	0.186515	0.759789	0.903703
	↪ 1.058237	1.204078	1.351928				
final_P80		v1_like	20.0	98.375436	0.731937	97.122506	97.692245
	↪ 98.273755	99.092960	99.701490				
final_R95		v1_like	20.0	83.550503	4.476638	74.806389	80.406333
	↪ 84.251968	86.321922	92.281284				

final_fines_frac	v1_like	20.0	0.090645	0.008823	0.073085	0.082795	↵
↵ 0.094473	0.096303	0.106333					
final_oversize_frac	v1_like	20.0	0.070904	0.007817	0.056652	0.065202	↵
↵ 0.073655	0.076033	0.083974					
final_R50_fines	v1_like	20.0	80.465424	4.221426	72.547138	78.085951	↵
↵81.112988	83.134031	89.232377					
final_R50_oversize	v1_like	20.0	36.595684	2.211801	31.481105	35.181110	↵
↵36.905044	38.204683	39.731812					
P80_ens_mean	v1_like	20.0	98.506232	0.028185	98.444504	98.491321	↵
↵98.505469	98.519237	98.559349					
P80_ens_std	v1_like	20.0	0.614661	0.131980	0.379865	0.540622	↵
↵ 0.615276	0.669631	0.956392					
R95_ens_mean	v1_like	20.0	82.388633	3.664841	76.325641	79.521573	↵
↵82.645993	85.768308	88.358322					
R95_ens_std	v1_like	20.0	3.911009	1.587328	1.398957	3.265352	↵
↵ 3.753038	4.432713	7.735091					
design_score	v1_like	20.0	5.252293	0.171618	4.939725	5.076281	↵
↵ 5.284991	5.387450	5.501329					
boundary_penalty	v1_like	20.0	0.665860	0.646062	0.000000	0.206611	↵
↵ 0.386396	1.000000	2.000000					
robust_design_score	v1_like	20.0	7.249871	1.965659	4.939725	5.984768	↵
↵ 6.446268	8.205398	11.456923					
energy	robust	20.0	3.480955	0.436551	2.840497	3.150288	↵
↵ 3.401452	3.809334	4.360648					
angle_rad	robust	20.0	0.665976	0.100897	0.508049	0.591191	↵
↵ 0.658301	0.709384	0.859783					
coupling	robust	20.0	1.047991	0.296623	0.601921	0.794166	↵
↵ 1.067815	1.278720	1.513037					
strength	robust	20.0	1.628959	0.407144	0.901115	1.377325	↵
↵ 1.721917	1.875561	2.321265					
porosity	robust	20.0	0.294325	0.013212	0.269562	0.283377	↵
↵ 0.292769	0.304983	0.314071					
gravity	robust	20.0	9.899087	0.120953	9.490150	9.858123	↵
↵ 9.900343	9.967632	10.083053					
atmosphere	robust	20.0	0.654963	0.133681	0.394378	0.577477	↵
↵ 0.657825	0.743823	0.867710					
shape_factor	robust	20.0	1.116853	0.147471	0.805102	1.040436	↵
↵ 1.126143	1.220025	1.333853					
final_P80	robust	20.0	98.367436	0.672776	97.122506	97.910286	↵
↵98.366818	98.811769	99.620259					
final_R95	robust	20.0	85.308963	5.339209	72.894158	82.040210	↵
↵85.531416	88.511947	94.419735					
final_fines_frac	robust	20.0	0.088760	0.007918	0.075938	0.080908	↵
↵ 0.089035	0.094567	0.103330					
final_oversize_frac	robust	20.0	0.069189	0.008672	0.056652	0.064462	↵
↵ 0.068728	0.071136	0.095883					

final_R50_fines	robust	20.0	82.467767	5.725052	69.845416	79.578712	↵
↵81.765952	86.779135	93.566750					
final_R50_oversize	robust	20.0	37.899871	3.182628	32.737757	36.060122	↵
↵37.425662	40.611598	43.326971					
P80_ens_mean	robust	20.0	98.488426	0.039226	98.414957	98.468127	↵
↵98.487830	98.504576	98.555360					
P80_ens_std	robust	20.0	0.728900	0.222095	0.363311	0.553302	↵
↵0.721932	0.901916	1.127328					
R95_ens_mean	robust	20.0	85.760477	5.257183	76.807189	81.476303	↵
↵86.298341	88.953392	95.556162					
R95_ens_std	robust	20.0	5.030097	3.728159	1.398957	3.369408	↵
↵3.966626	5.009105	16.184090					
design_score	robust	20.0	5.677475	0.338000	4.939725	5.420590	↵
↵5.764239	5.914741	6.110035					
boundary_penalty	robust	20.0	0.014400	0.040922	0.000000	0.000000	↵
↵0.000000	0.003286	0.181065					
robust_design_score	robust	20.0	5.720674	0.350503	4.939725	5.475045	↵
↵5.806141	6.009650	6.110035					

Robust selected designs:

	energy	angle_rad	coupling	strength	porosity	gravity	atmosphere	↵
	↵shape_factor	final_P80	final_R95	final_fines_frac	final_oversize_frac			↵
	↵final_R50_fines	\						
0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	0.486195	↵
	↵1.321979	98.139903	86.592805		0.095237		0.070884	↵
	↵81.897310							
1	2.846707	0.590529	1.080326	1.378950	0.282806	9.901995	0.683393	↵
	↵1.159860	97.934175	86.231628		0.089252		0.056652	↵
	↵83.240889							
2	3.857509	0.625831	0.795776	1.514135	0.309255	9.938662	0.841641	↵
	↵1.181244	98.410902	72.894158		0.080413		0.070905	↵
	↵69.845416							
3	2.840497	0.686096	0.841353	1.134742	0.283432	9.943441	0.867710	↵
	↵0.953802	97.122506	84.213834		0.086094		0.058068	↵
	↵81.634593							
4	3.547463	0.693527	1.513037	2.183828	0.291489	9.972195	0.663254	↵
	↵1.069653	97.696331	83.201911		0.100264		0.073963	↵
	↵78.870930							
5	3.600797	0.511221	1.264328	1.922918	0.303837	10.008936	0.617995	↵
	↵1.245303	98.181368	85.413788		0.096042		0.075254	↵
	↵82.142556							
6	3.287377	0.851607	0.624994	1.003748	0.269562	9.897003	0.624273	↵
	↵1.219199	99.514990	80.321074		0.078780		0.068611	↵
	↵79.842654							

7	3.870405	0.591411	0.789338	1.501928	0.313308	9.784023	0.836029	┘
	↪ 0.805102	98.322733	78.754366		0.080917		0.071829	┘
	↪ 76.982888							
8	2.947683	0.508049	1.448914	1.767410	0.290708	9.937267	0.652395	┘
	↪ 1.170228	97.537812	81.666734		0.103330		0.070443	┘
	↪ 78.065900							
9	3.155230	0.780296	1.306914	1.752125	0.302273	10.014013	0.767409	┘
	↪ 0.924279	98.879083	94.419735		0.091754		0.068113	┘
	↪ 93.566750							
10	2.988285	0.556420	0.601921	0.901115	0.280882	9.875373	0.397764	┘
	↪ 1.222503	97.838616	88.132527		0.080879		0.095883	┘
	↪ 86.505068							
11	3.380189	0.637083	0.868310	1.372452	0.283213	9.884139	0.766936	┘
	↪ 1.079954	98.573698	82.164701		0.083960		0.060725	┘
	↪ 79.814639							
12	3.386214	0.665088	1.275168	1.824141	0.286388	9.966110	0.571077	┘
	↪ 1.088572	98.665508	85.649044		0.094344		0.067753	┘
	↪ 80.214266							
13	3.135462	0.665701	0.623075	1.024844	0.300897	10.003710	0.625695	┘
	↪ 1.322433	98.522520	88.254146		0.075938		0.063752	┘
	↪ 87.601335							
14	4.176588	0.755998	0.978929	1.869876	0.294049	9.847830	0.561900	┘
	↪ 1.000532	98.914848	93.971189		0.086300		0.065575	┘
	↪ 91.353592							
15	3.369114	0.859783	1.155627	1.712666	0.311486	9.898691	0.579610	┘
	↪ 0.902234	98.246032	83.745989		0.093293		0.064699	┘
	↪ 80.233954							
16	3.639483	0.693846	1.055303	1.731167	0.290888	10.083053	0.394378	┘
	↪ 1.333853	98.789331	89.285351		0.090528		0.069038	┘
	↪ 87.668191							
17	4.360648	0.638533	1.152940	2.231277	0.314071	9.835256	0.723607	┘
	↪ 1.053737	99.620259	92.096781		0.088818		0.068845	┘
	↪ 88.436295							
18	3.793276	0.572610	1.506900	2.321265	0.274615	9.838338	0.701890	┘
	↪ 1.190159	97.349092	79.676362		0.099352		0.081730	┘
	↪ 75.163744							
19	4.019477	0.651513	0.787290	1.537973	0.294915	9.490150	0.736119	┘
	↪ 1.092427	99.089018	89.493133		0.079706		0.061057	┘
	↪ 86.274362							
	final_R50_oversize	P80_ens_mean	P80_ens_std	R95_ens_mean	R95_ens_std			┘
	↪ design_score	boundary_penalty	robust_design_score					
0		37.264893	98.492224	0.389112	86.913264	3.349563		┘
	↪ 4.939725		0.000000	4.939725				
1		38.084762	98.483203	0.572645	86.495709	1.398957		┘
	↪ 5.078469		0.000000	5.078469				

2	32.737757	98.472878	0.722666	76.807189	3.376023	␣
↪ 5.279804	0.000000		5.279804			
3	39.731812	98.444504	0.682039	78.768877	4.163790	␣
↪ 5.352436	0.000000		5.352436			
4	35.577029	98.555360	0.547707	80.294774	5.013393	␣
↪ 5.386258	0.000000		5.386258			
5	36.545196	98.506245	0.719625	85.285994	4.275933	␣
↪ 5.432034	0.024202		5.504640			
6	37.586431	98.474173	0.895484	83.011685	3.564213	␣
↪ 5.670350	0.000000		5.670350			
7	33.916143	98.464169	0.363311	87.476221	16.184090	␣
↪ 5.752111	0.000000		5.752111			
8	33.166536	98.491711	0.949540	88.635690	3.769461	␣
↪ 5.776367	0.000000		5.776367			
9	43.326971	98.447098	0.501512	92.649423	4.645849	␣
↪ 5.698344	0.034949		5.803191			
10	40.460563	98.483949	1.127328	79.793245	5.007676	␣
↪ 5.809091	0.000000		5.809091			
11	36.912677	98.539838	0.921211	83.621733	3.711544	␣
↪ 5.869888	0.000000		5.869888			
12	36.389423	98.552938	0.768263	86.229968	4.765755	␣
↪ 5.892969	0.000000		5.892969			
13	42.141769	98.546248	0.979481	81.870147	3.537855	␣
↪ 5.882616	0.013143		5.922045			
14	41.984183	98.430352	0.555167	91.589942	2.843591	␣
↪ 6.002598	0.000000		6.002598			
15	36.221153	98.499001	1.031247	89.906498	5.146537	␣
↪ 6.030805	0.000000		6.030805			
16	39.861756	98.504020	0.802632	86.366715	2.544662	␣
↪ 5.520460	0.181065		6.063654			
17	41.064703	98.496205	0.721198	93.748116	6.146061	␣
↪ 5.980057	0.034634		6.083959			
18	33.942326	98.414957	0.892798	80.188191	2.483948	␣
↪ 6.085082	0.000000		6.085082			
19	41.081336	98.469447	0.435032	95.556162	14.673043	␣
↪ 6.110035	0.000000		6.110035			

1.14 14. Bounds Check

```
[16]: def bounds_check_table(selected_df, name):
    rows = []
    for col in input_cols:
        rows.append({
            "selection": name,
            "feature": col,
            "selected_min": selected_df[col].min(),
```

```

        "selected_max": selected_df[col].max(),
        "allowed_min": input_bounds[col]["min"],
        "allowed_max": input_bounds[col]["max"],
        "inside_bounds": (
            selected_df[col].min() >= input_bounds[col]["min"]
            and selected_df[col].max() <= input_bounds[col]["max"]
        ),
        "min_distance_to_lower": selected_df[col].min() -
↪input_bounds[col]["min"],
        "min_distance_to_upper": input_bounds[col]["max"] -
↪selected_df[col].max(),
    })
    return pd.DataFrame(rows)

bounds_compare = pd.concat([
    bounds_check_table(selected_v1_like, "v1_like"),
    bounds_check_table(selected_robust, "robust"),
], ignore_index=True)

display(bounds_compare)

```

	selection	feature	selected_min	selected_max	allowed_min	allowed_max
↪	inside_bounds	min_distance_to_lower		min_distance_to_upper		
0	v1_like	energy	2.840497	4.373093	0.500000	5.000000
↪		True	2.340497		0.626907	
1	v1_like	angle_rad	0.501950	0.827086	0.261799	1.570796
↪		True	0.240151		0.743710	
2	v1_like	coupling	0.545276	1.612882	0.200000	1.700000
↪		True	0.345276		0.087118	
3	v1_like	strength	1.134742	2.358284	0.400000	4.200000
↪		True	0.734742		1.841716	
4	v1_like	porosity	0.271333	0.330000	0.000000	0.330000
↪		True	0.271333		0.000000	
5	v1_like	gravity	9.861554	10.470000	1.020000	10.470000
↪		True	8.841554		0.000000	
6	v1_like	atmosphere	0.445090	0.872075	0.000000	1.000000
↪		True	0.445090		0.127925	
7	v1_like	shape_factor	0.759789	1.351928	0.700000	1.500000
↪		True	0.059789		0.148072	
8	robust	energy	2.840497	4.360648	0.500000	5.000000
↪		True	2.340497		0.639352	
9	robust	angle_rad	0.508049	0.859783	0.261799	1.570796
↪		True	0.246250		0.711014	
10	robust	coupling	0.601921	1.513037	0.200000	1.700000
↪		True	0.401921		0.186963	
11	robust	strength	0.901115	2.321265	0.400000	4.200000
↪		True	0.501115		1.878735	

12	robust	porosity	0.269562	0.314071	0.000000	0.330000
	↪	True	0.269562		0.015929	
13	robust	gravity	9.490150	10.083053	1.020000	10.470000
	↪	True	8.470150		0.386947	
14	robust	atmosphere	0.394378	0.867710	0.000000	1.000000
	↪	True	0.394378		0.132290	
15	robust	shape_factor	0.805102	1.333853	0.700000	1.500000
	↪	True	0.105102		0.166147	

1.15 15. Save Robust Refined Design Submission

The final required file contains only the input parameters and `submission_id`.

```
[17]: final_selected = selected_robust.copy()

# If robust selection failed for any reason, fallback to v1-like selection.
if len(final_selected) < 20:
    final_selected = selected_v1_like.copy()

final_selected = final_selected.head(20).reset_index(drop=True)

design_submission = pd.DataFrame({"submission_id": np.
    ↪ arange(len(final_selected))})
for col in input_cols:
    design_submission[col] = final_selected[col].values

design_submission_path = SUBMISSIONS_DIR / "design_submission_robust_refined.
    ↪ csv"
design_submission.to_csv(design_submission_path, index=False)

selected_diagnostics_path = REPORTS_DIR / ↪
    ↪ "inverse_design_selected_robust_refined_diagnostics.csv"
final_selected.to_csv(selected_diagnostics_path, index=False)

print("Saved robust refined design submission to:", design_submission_path)
print("Saved diagnostics to:", selected_diagnostics_path)
print("Shape:", design_submission.shape)
display(design_submission)
```

```
Saved robust refined design submission to: /home/alouiyaz/projects/boom-
challenge-ejecta-
prediction/outputs/submissions/design_submission_robust_refined.csv
Saved diagnostics to: /home/alouiyaz/projects/boom-challenge-ejecta-
prediction/reports/inverse_design_selected_robust_refined_diagnostics.csv
Shape: (20, 9)
```

	submission_id	energy	angle_rad	coupling	strength	porosity	gravity
	↪ atmosphere	shape_factor					

0		0	3.416691	0.784376	1.289377	1.892615	0.308421	9.861554	↵
↵	0.486195		1.321979						
1		1	2.846707	0.590529	1.080326	1.378950	0.282806	9.901995	↵
↵	0.683393		1.159860						
2		2	3.857509	0.625831	0.795776	1.514135	0.309255	9.938662	↵
↵	0.841641		1.181244						
3		3	2.840497	0.686096	0.841353	1.134742	0.283432	9.943441	↵
↵	0.867710		0.953802						
4		4	3.547463	0.693527	1.513037	2.183828	0.291489	9.972195	↵
↵	0.663254		1.069653						
5		5	3.600797	0.511221	1.264328	1.922918	0.303837	10.008936	↵
↵	0.617995		1.245303						
6		6	3.287377	0.851607	0.624994	1.003748	0.269562	9.897003	↵
↵	0.624273		1.219199						
7		7	3.870405	0.591411	0.789338	1.501928	0.313308	9.784023	↵
↵	0.836029		0.805102						
8		8	2.947683	0.508049	1.448914	1.767410	0.290708	9.937267	↵
↵	0.652395		1.170228						
9		9	3.155230	0.780296	1.306914	1.752125	0.302273	10.014013	↵
↵	0.767409		0.924279						
10		10	2.988285	0.556420	0.601921	0.901115	0.280882	9.875373	↵
↵	0.397764		1.222503						
11		11	3.380189	0.637083	0.868310	1.372452	0.283213	9.884139	↵
↵	0.766936		1.079954						
12		12	3.386214	0.665088	1.275168	1.824141	0.286388	9.966110	↵
↵	0.571077		1.088572						
13		13	3.135462	0.665701	0.623075	1.024844	0.300897	10.003710	↵
↵	0.625695		1.322433						
14		14	4.176588	0.755998	0.978929	1.869876	0.294049	9.847830	↵
↵	0.561900		1.000532						
15		15	3.369114	0.859783	1.155627	1.712666	0.311486	9.898691	↵
↵	0.579610		0.902234						
16		16	3.639483	0.693846	1.055303	1.731167	0.290888	10.083053	↵
↵	0.394378		1.333853						
17		17	4.360648	0.638533	1.152940	2.231277	0.314071	9.835256	↵
↵	0.723607		1.053737						
18		18	3.793276	0.572610	1.506900	2.321265	0.274615	9.838338	↵
↵	0.701890		1.190159						
19		19	4.019477	0.651513	0.787290	1.537973	0.294915	9.490150	↵
↵	0.736119		1.092427						

1.16 16. Optional: Also Save as Official design_submission.csv

Uncomment this cell if you want this robust refined version to be the official inverse-design file.

```
[18]: # official_path = SUBMISSIONS_DIR / "design_submission.csv"
      # design_submission.to_csv(official_path, index=False)
      # print("Saved official design submission to:", official_path)
```

1.17 17. Final Interpretation

Use this section to write notes after execution:

- Did the boundary penalty reduce candidates close to upper/lower bounds?
- Did `P80_ens_mean` remain safely inside 96–101?
- Did `R95_ens_mean` remain far below 175?
- Did the selected designs remain diverse?
- Are all inputs inside the allowed constraints?

If the robust selection is too conservative, reduce `boundary_weight` from 3.0 to 1.0 or 2.0.

If it is still too close to bounds, increase `boundary_weight` to 5.0.